

Challenges

Information Flow Challenge 1-5

IFC 1

Since any syntactically valid program is accepted, we can simply assign `h` to `l`.

IFC Step 1

Task: Ensure that at any runs of your program, the low (public) variable `l` has the same value as the high (private) variable `h`.

All variables are boolean and can therefore only be **true** or **false**.

Write in your program in the text area below, satisfying the following type system.

A variable is *high* if its name starts with the letter "h", otherwise it is *low*.

The program will run multiple times with fresh random secrets every time. All runs must leak successfully.

1

`l = h;`

Submit

Parsing...
Typechecking...
Passed 20/20. Great job!

Congrats, you've leaked the secret correctly. Code: codfish

IFC 2

All variables are boolean, meaning they can only be true or false. Without direct assignment, we can use an if-else statement to determine the value of `h` and leak the corresponding value to `l`.

IFC Step 2

Task: Ensure that at any runs of your program, the low (public) variable `l` has the same value as the high (private) variable `h`.

All variables are boolean and can therefore only be **true** or **false**.

Write in your program in the text area below, satisfying the following type system.

A variable is *high* if its name starts with the letter "h", otherwise it is *low*.

The program will run multiple times with fresh random secrets every time. All runs must leak successfully.

```
1 if (h) {
2   l = true;
3 } else {
4   l = false;
5 }
```

Submit

Parsing...
Typechecking...
Passed 20/20. Great job!

Congrats, you've leaked the secret correctly. Code: joystick

IFC 3

Since both `hatch` and `h` are high-security variables, we can assign `h` to `hatch`. By the typing rule, `declassify(hatch)` yields a low-security value, so we can safely assign it to `l` and thereby leak the secret `h`.

IFC Step 3

Task: Ensure that at any runs of your program, the low (public) variable `l` has the same value as the high (private) variable `h`.

All variables are boolean and can therefore only be **true** or **false**.

Write in your program in the text area below, satisfying the following type system.

A variable is *high* if its name starts with the letter "h", otherwise it is *low*.

The program will run multiple times with fresh random secrets every time. All runs must leak successfully.

```
1 hatch = h;
2 l = declassify(hatch);
```

Submit

Parsing...
Typechecking...
Passed 20/20. Great job!

Congrats, you've leaked the secret correctly. Code: graphite

IFC 4

The assignment made by a let block is always accepted. So the local variable `k` takes the value of `h`, and assigning `k` to `l` is not restricted.

IFC Step 4

Task: Ensure that at any runs of your program, the low (public) variable `l` has the same value as the high (private) variable `h`.

All variables are boolean and can therefore only be **true** or **false**.

Write in your program in the text area below, satisfying the following type system.

A variable is *high* if its name starts with the letter "h", otherwise it is *low*.

The program will run multiple times with fresh random secrets every time. All runs must leak successfully.

```
1 let (k = h) in
2   l = k;
```

Submit

Parsing...

Typechecking...

Passed 20/20. Great job!

Congrats, you've leaked the secret correctly. Code: allergy

IFC 5

Like in step 2, we can use an if statement to check the value of `h`. However, we cannot catch a thrown exception to assign a value to `l` because low variables cannot be assigned within a high block. Therefore, by throwing an exception, execution is interrupted, ensuring that `l` has the same value as `h`.

IFC Step 5

Task: Ensure that at any runs of your program, the low (public) variable **l** has the same value as the high (private) variable **h**.

All variables are boolean and can therefore only be **true** or **false**.

Write in your program in the text area below, satisfying the following type system.

A variable is *high* if its name starts with the letter "h", otherwise it is *low*.

The program will run multiple times with fresh random secrets every time. All runs must leak successfully.

```
1 l = true;
2 try {
3   if (h) {
4     throw;
5   }
6   l = false;
7 }
8 catch {
9   skip;
10 }
```

Submit

Parsing...
Typechecking...
Passed 20/20. Great job!

Congrats, you've leaked the secret correctly. Code: collect

Beneri

5. No .php files here!

5. No .php files here!

Picking a good photo for your dating profile is tricky. You need to think about lightning, background, etc. But perhaps most important is **the file extension**. A classic **.jpg**? A transparent **.png**? Or maybe a funny **.gif**? Perhaps a **.php** file? 🐱

Well, this site only allows a few extensions, as can be seen in the validation code below:

```
$filename = basename($_FILES["file"]["name"]);
$ext = explode(".", $filename)[1];

if( !($ext == "jpg" || $ext == "png" || $ext == "gif") ) {
    die("Only jpg, png and gif are allowed!");
}
```

Can you upload a file with the **.php** file extension, that also executes the function `win()`? I.e. the file should contain `<?php win(); ?>`.

Correct!

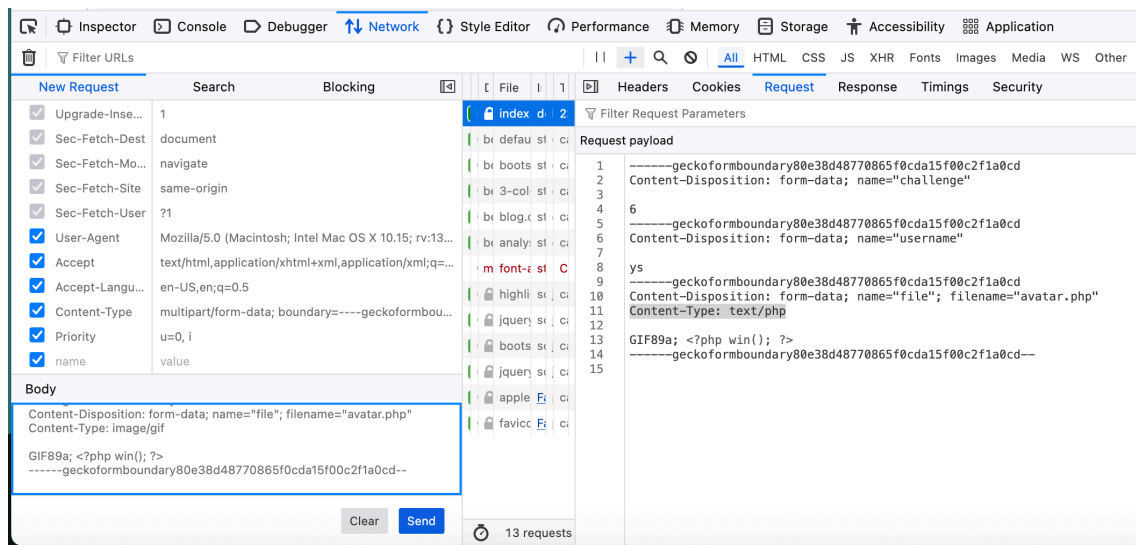
Solution: upload a file named `avatar.jpg.php` which contains `<?php win(); ?>`.

The validator naively splits the filename on dots and only inspects the part immediately after the first dot, so in a name like `avatar.jpg.php`, it sees "jpg" and allows the upload, even though

the true extension is "php" and the server will still execute the PHP code. Because it only checks the first extension fragment, we can hide a disallowed suffix behind a permitted one.

6. You're not my type.

The vulnerability exists because the server only checks the MIME type (`$_FILES['file']['type']`) without verifying the actual file content or extension. To exploit this weakness, we create a PHP script that begins with the GIF header signature (`GIF89a;`) followed by the PHP payload `<?php win(); ?>` . When the script is submitted via the upload form, the initial request shows the file part labeled as `Content-Type: text/php` . By opening Firefox's Network tab, locating the POST request, and choosing "Edit and Resend," we can alter/modify the Content-Type header to "image/jpeg" before resubmission. The server accepts the upload as an image file but the web server still processes it as PHP code due to its .php extension. As a result, accessing the uploaded file triggers execution of the `win()` function.



6. You're not my type.

After breaking the extension check in the previous challenge, the developers decided to skip the extension check and instead go for a solid and secure `type` check.

The code below is used to block any non-image upload:

```
$type = $_FILES['file']['type'];
if( !($type == "image/jpeg" || $type == "image/png" || $type == "image/gif") ) {
    die("Wrong file type, only images allowed!");
}
```

Can you upload a file with the .php file extension, that also executes the function `win()` ? I.e. the file should contain `<?php win(); ?>` .

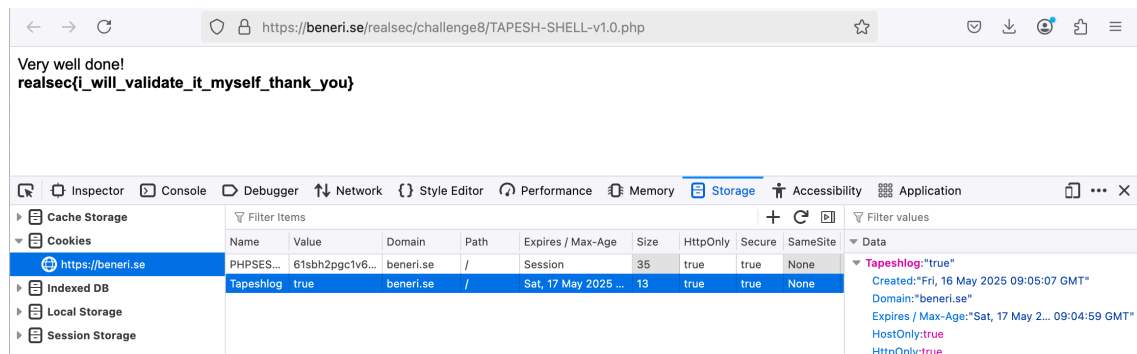
Correct!

8. Crack the shell.

By reading the code, we can see that "Tapeshlog" is the key to accessing the shell. The authentication mechanism uses two cookies: "TapeshPassword" and "Tapeshlog". The code first checks if a password submitted through a form matches the expected hash stored in the "TapeshPassword" cookie. If this check passes, it sets "Tapeshlog" to "true" which grants access

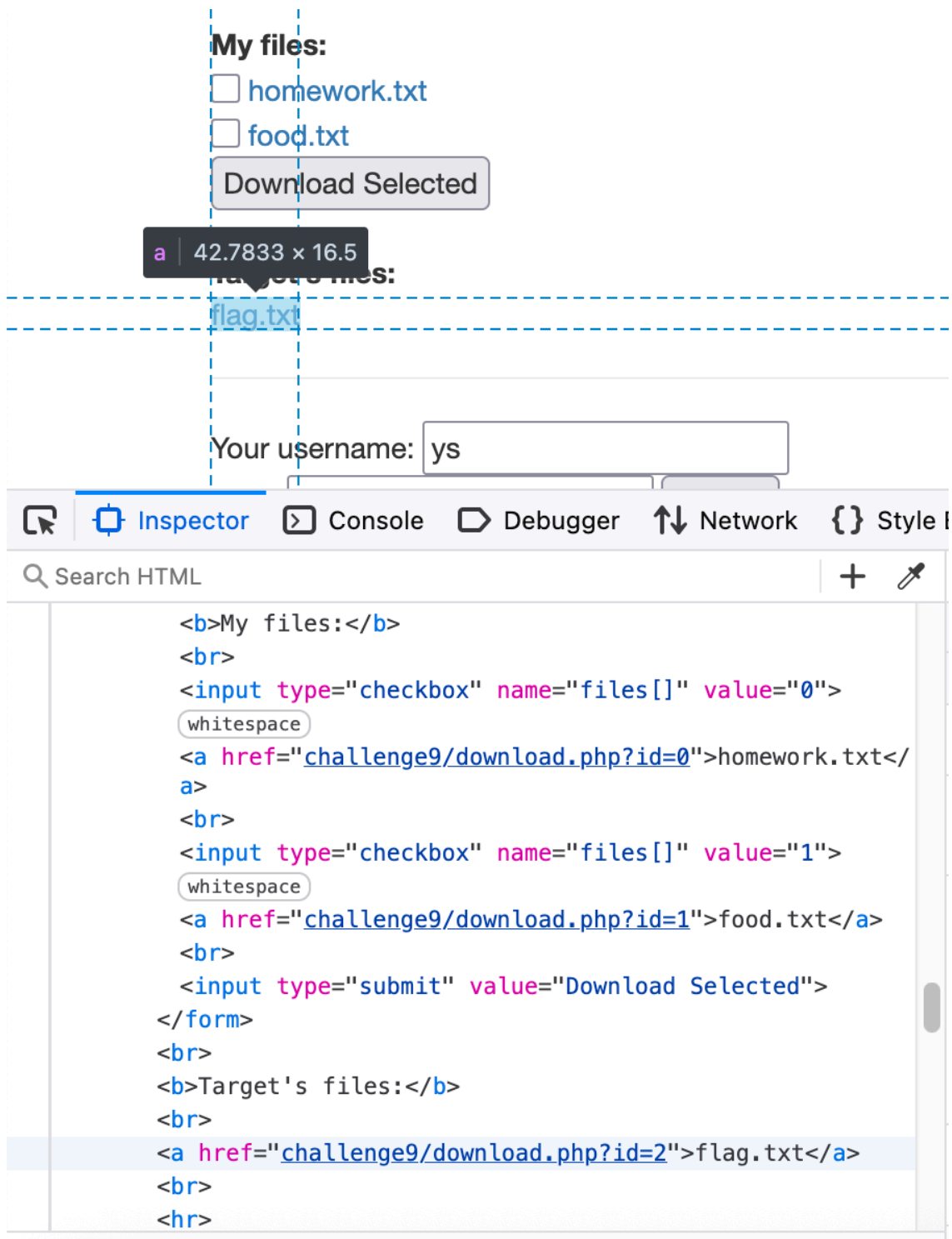
to the shell. The critical vulnerability here is that once "Tapeslog" is set to "true", the application never verifies the password again. It simply checks if the "Tapeslog" cookie exists and is not set to "false". This means we can bypass the entire password verification process by manually setting this cookie ourselves, without needing to know the original password.

Using Firefox's inspect feature, we simply add a new cookie named "Tapeslog" with the value "true" for the website's domain. After opening the `TAPESH-SHELL-v1.0.php` with our newly created cookie in place, we gain immediate access to the flag. The authentication check is completely bypassed, and we can now see the flag: `realsec{i_will_validate_it_myself_thank_you}`.



9. It takes two to tango.

When opening Firefox Inspector to examine the page's HTML structure, we can see that the `flag.txt` file is actually referenced through an `href` link pointing to `"challenge9/download.php?id=2"`. This suggested that files are being accessed through an ID parameter rather than direct file paths.



When analyzing the form responsible for downloading selected files, we noticed it uses a POST request to "challenge9/download.php" with an array parameter named "files[]" containing the selected file IDs (0 for homework.txt and 1 for food.txt). To exploit this vulnerability clearly, we can modify the form by adding a new checkbox element for the flag.txt file. In Firefox Inspector, add a new input element: ☐. After making this modification, a new checkbox appeared next to download button.

My files:

☐ homework.txt

☐ food.txt

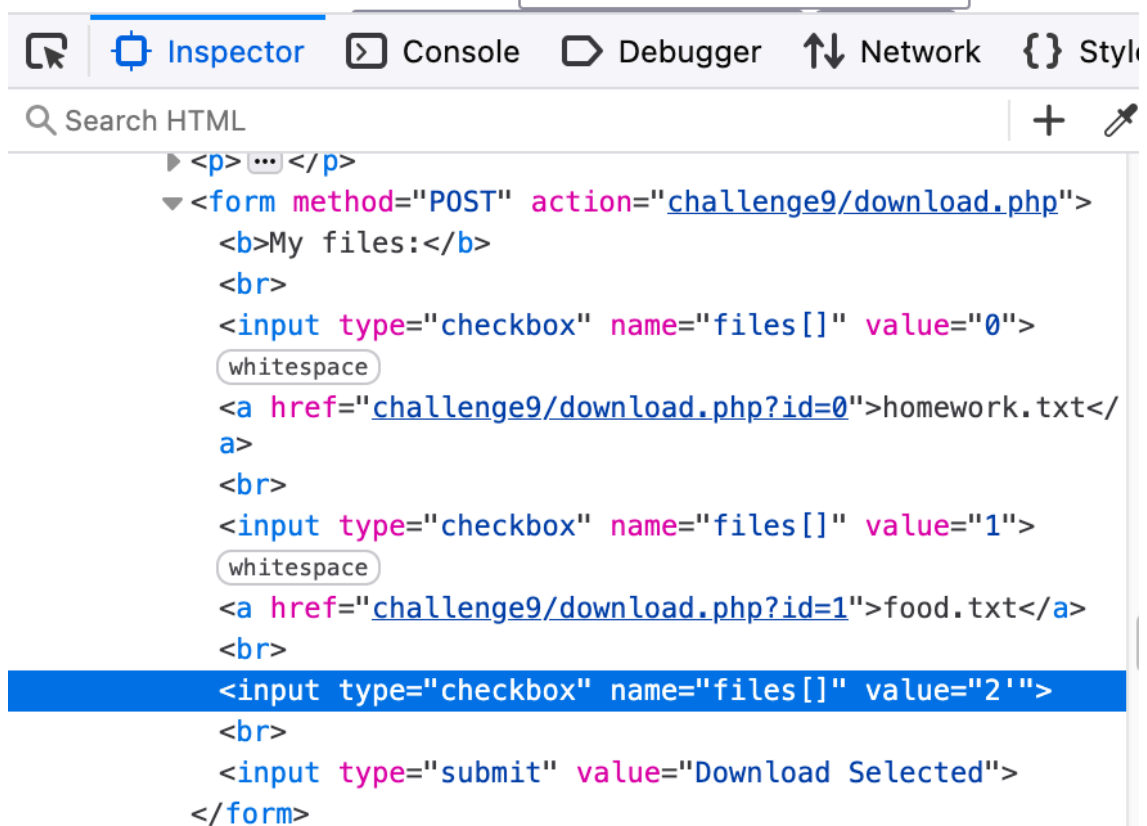


Download Selected

Target's files:

flag.txt

Your username:



Then select this newly created checkbox and click "Download Selected." As a result, we successfully downloaded the zip of flag.txt, revealing the flag in the format `realsec{zip_it_i_own_this_file}`.